Laboratorios
Docker MQTT-Postgres
TLS-Compose-Docker

Salomé Zuluaga Sánchez, Sara Escobar Sánchez

Facultad de Ingeniería, Universidad Autónoma de Manizales
Manizales, Colombia

I. INTRODUCCIÓN Y OBJETIVOS

El uso de contenedores ha transformado la forma en que se desarrollan, implementan y mantienen las aplicaciones, ya que permite ejecutar servicios de manera aislada y facilitar su configuración en diferentes entornos. En este contexto, Docker es una plataforma de código abierto que permite crear, ejecutar y administrar contenedores, ofreciendo una alternativa ligera y flexible para desplegar aplicaciones sin necesidad de utilizar máquinas virtuales completas. Su funcionamiento se basa en características del kernel de Linux, como los grupos de control (*cgroups*) y los espacios de nombres (*namespaces*), que permiten administrar los recursos del sistema y mantener separados los procesos que se ejecutan en cada contenedor [1].

En esta práctica se aplican estos conceptos mediante la implementación de una canalización de datos para un sistema de Internet de las Cosas (IoT). Para ello, se configuran tres servicios conectados a través de una red virtual de tipo *bridge*: un broker MQTT Eclipse Mosquitto, una base de datos PostgreSQL 18 y un cliente desarrollado en Python. Estos servicios permiten recibir datos de temperatura mediante el protocolo MQTT, basado en un modelo de publicación y suscripción, y almacenarlos en la base de datos utilizando las bibliotecas Paho-MQTT y psycopg2.

OBJETIVO GENERAL

Construir y validar de extremo a extremo una canalización de datos IoT sobre Docker, que reciba lecturas de temperatura por MQTT y las almacene de forma persistente en PostgreSQL.

OBJETIVOS ESPECÍFICOS

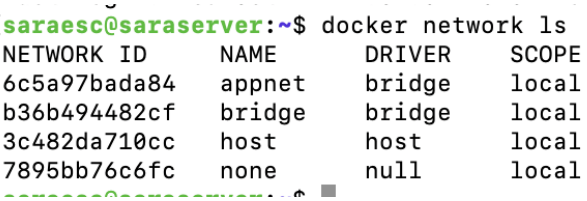
- Implementar un entorno de servicios contenerizados con Docker, configurando un broker MQTT con autenticación, una base de datos PostgreSQL 18 y una red bridge para permitir la comunicación entre los servicios.
- Desarrollar y contenerizar un cliente en Python que reciba mensajes MQTT en formato JSON, procese los datos de temperatura y los almacene en tablas de PostgreSQL, garantizando la persistencia de la información.
- Verificar el funcionamiento de la dirección de datos mediante la publicación de mensajes de prueba, la inspección de los registros almacenados y la identificación de posibles errores de comunicación, autenticación y persistencia.

II. Docker MQTT-Postgres
METODOLOGÍA

Para el desarrollo e implementación de esta arquitectura IoT orientada a la recolección de datos de sensores, estructuramos el trabajo en contenedores Docker, lo que nos permitió aislar los servicios y asegurar que se comunicaran de forma privada.

1. Preparación del entorno y estructura del proyecto

Lo primero fue definir una red tipo "bridge" definida por el usuario llamada appnet,una red bridge de Docker es una red virtual privada que se crea dentro del propio equipo y a la que se conectan los contenedores,estos unidos a una misma red bridge pueden comunicarse libremente entre sí.
Esto fue clave para que nuestros contenedores pudieran encontrarse por nombre en lugar de depender de direcciones IP estáticas. Organizamos los directorios del proyecto separando las carpetas para el broker, la base de datos y el código del cliente Python. También centralizamos las credenciales en un archivo .env para no dejar contraseñas expuestas de forma insegura.
Se verificó la existencia del puente con el listado de redes.



Img 1. Listado de redes en el docker

2. Despliegue del broker Mosquitto

Configuramos el broker MQTT para recibir los datos de temperatura. Para hacerlo más seguro, desactivamos las conexiones anónimas desde el archivo mosquitto.conf y generamos un archivo de contraseñas encriptadas usando la herramienta interna mosquitto_passwd. Luego, levantamos el contenedor mapeando el puerto 1883 hacia el host para permitir que dispositivos externos puedan publicar, y montamos volúmenes locales para no perder la configuración ni los logs si el contenedor se apaga.

Para comprobar que nuestra configuración quedó correctamente aplicada, realizamos una prueba de inyección. Primero, verificamos en los registros del contenedor que el servicio iniciara sin errores, observando el mensaje mosquitto version 2.1.2 running. Inmediatamente después, forzamos la publicación de un mensaje de prueba ("hi") en un tópico cualquiera (test) omitiendo intencionalmente el envío de credenciales. Como esperábamos, el broker interceptó la petición y la rechazó, arrojando el error Connection error: Connection Refused: not authorised. Esto nos dio la certeza de que nuestro nodo central está protegido contra publicaciones maliciosas o no identificadas.

```
1792615963: mosquitto version 2.1.2 starting
1792615963: Config loaded from /mosquitto/config/mosquitto.conf.
1792615963: Bridge support available.
1792615963: Persistence support available.
1792615963: TLS support available.
1792615963: TLS-PSK support available.
1792615963: Websockets support available.
1792615963: Plugin builtin-security has registered to receive 'basic-auth' events.
1792615963: Opening ipv4 listen socket on port 1883.
1792615963: mosquitto version 2.1.2 running
1792615963: New connection from 172.18.0.3:43550 on port 1883.
1792615963: Client 172.18.0.3 [172.18.0.3:43550] disconnected: connection closed by client.
1792615965: New connection from 172.18.0.3:43560 on port 1883.
1792615965: Client 172.18.0.3 [172.18.0.3:43560] disconnected: connection closed by client.
1792615965: New connection from 172.18.0.3:33411 on port 1883.
1792615965: New client connected from 172.18.0.3:33411 as pg-client (p5, c1, k60, u'admin').
Connection error: Connection Refused: not authorised
Error: The connection was refused
saraesc@saraserver:~$
```

Img 2. Prueba de inyección en mosquitto

3. Despliegue de PostgreSQL 18

Para el almacenamiento de los datos, configuramos un contenedor con PostgreSQL en su versión 18. Básicamente, PostgreSQL es un motor de base de datos relacional y de código abierto, que en nuestra arquitectura cumple el rol de ser el destino final de la información. Esto es importante en los resultados para estructurar y guardar de forma permanente cada lectura de temperatura que procese el sistema, asociando cada dato con un identificador único (ID) y la estampa de tiempo exacta de su recepción (received_at), lo cual es vital para tener un registro ordenado y poder hacer consultas futuras sobre el historial de telemetría.

Al levantar este contenedor, y a diferencia del broker, decidimos no exponer el puerto de la base de datos (5432) al host por seguridad; solo los contenedores que estén dentro de nuestra red appnet pueden interactuar con ella. Además, mapeamos un volumen local en /var/lib/postgresql para garantizar la persistencia física de los registros, evitando que el historial de los sensores se pierda si el contenedor llega a reiniciarse o apagarse.

```
[saraesc@saraserver:~$ cd iot-lab
[saraesc@saraserver:~/iot-lab$ ls
 LICENSE  README.md  broker  client  database
[saraesc@saraserver:~/iot-lab$ cd database
[saraesc@saraserver:~/iot-lab/database$ ls
 pgdata
[saraesc@saraserver:~/iot-lab/database$ cd pgdata
[saraesc@saraserver:~/iot-lab/database/pgdata$ ls
 18
```

Img 3.Base de datos PostgreSQL versión 18

4. Desarrollo y contenerización del cliente

Para lograr que los datos fluyeran de forma autónoma desde el broker hasta la base de datos, implementamos un cliente a medida. Este desarrollo se dividió en la lógica principal, la gestión del arranque y su empaquetamiento:

a. Desarrollo del cliente principal (client.py):

Escribimos un script en Python utilizando la librería paho-mqtt en su versión 5 para suscribirnos al tópico sensors/temp. La lógica de este código está diseñada para interceptar los mensajes y decodificar su carga útil asumiendo que vienen en formato JSON (por ejemplo, {"temp": 25.5}). Una de las características más prácticas que le implementamos fue la creación dinámica de tablas: cada vez que llega un mensaje, el código adapta el nombre del tópico (cambiando la barra / por un guion bajo _ para obtener sensors_temp) y verifica si la tabla existe en PostgreSQL. Si es la primera vez que se publica en ese tópico, el cliente crea automáticamente la tabla con sus respectivas columnas (un ID autoincremental, el valor de temperatura y la marca de tiempo received_at). Finalmente, el script inserta el valor abriendo una conexión rápida a la base de datos.

```
saraesc@saraserver:~/iot-lab/client$ cd src
saraesc@saraserver:~/iot-lab/client/src$ ls
client.py  wait_for.py
saraesc@saraserver:~/iot-lab/client/src$ cat client.py
import os
import json
import psycopg2
import paho.mqtt.client as mqtt

# Configuración desde el archivo .env
MQTT_HOST = os.environ["MQTT_HOST"]
MQTT_PORT = int(os.environ["MQTT_PORT"])
MQTT_USER = os.environ["MQTT_USER"]
MQTT_PASSWORD = os.environ["MQTT_PASSWORD"]
MQTT_TOPIC = os.environ["MQTT_TOPIC"]

PG_HOST = os.environ.get("POSTGRES_HOST", "postgres")
PG_PORT = os.environ.get("POSTGRES_PORT", "5432")
PG_DB = os.environ["POSTGRES_DB"]
PG_USER = os.environ["POSTGRES_USER"]
PG_PASSWORD = os.environ["POSTGRES_PASSWORD"]

def table_name_for_topic(topic: str) -> str:
    return topic.strip("/").replace("/", "_")

def ensure_table(cur, table: str):
    cur.execute(f"""
        CREATE TABLE IF NOT EXISTS {table} (
            id SERIAL PRIMARY KEY,
            temp DOUBLE PRECISION,
            received_at TIMESTAMPTZ DEFAULT NOW()
        );
    """)

def insert_reading(cur, table: str, payload: dict):
    cur.execute(
        f'INSERT INTO "{table}" (temp) VALUES (%s)',
        (payload.get("temp"),)
    )

def on_connect(client, userdata, flags, rc, properties=None):
    print(f"[mqtt] connected rc={rc}, subscribing to {MQTT_TOPIC}")
    client.subscribe(MQTT_TOPIC)

def on_message(client, userdata, msg):
    topic = msg.topic
    table = table_name_for_topic(topic)
    try:
        payload = json.loads(msg.payload.decode())
```

Img 4.Codigo del cliente

b. Gestión de dependencias en el arranque (wait_for.py):

Al trabajar en un entorno con múltiples contenedores que inician al mismo tiempo, nos enfrentamos al problema de que el cliente de Python podía intentar conectarse a la base de datos o al broker antes de que estos estuvieran totalmente listos. Para evitar errores de conexión al arrancar, desarrollamos un script auxiliar llamado wait_for.py. Utilizando únicamente la librería estándar de Python (socket), este código se encarga de "hacer ping" a los puertos TCP de Mosquitto (1883) y PostgreSQL (5432). El script se queda en un ciclo de espera y solo cede la ejecución al cliente principal (client.py) cuando comprueba que ambos servicios están respondiendo exitosamente.

```
saraesc@saraserver:~/iot-lab/client/src$ cat wait_for.py
import os
import socket
import time
import sys

DEPS = [
    ("postgres", os.environ.get("POSTGRES_HOST", "postgres"), int(os.environ.get("POSTGRES_PORT", "5432"))),
    ("mosquitto", os.environ.get("MQTT_HOST", "mosquitto"), int(os.environ.get("MQTT_PORT", "1883"))),
]

TIMEOUT = int(os.environ.get("WAIT_TIMEOUT", "120"))
INTERVAL = int(os.environ.get("WAIT_INTERVAL", "2"))

def reachable(host: str, port: int) -> bool:
    try:
        with socket.create_connection((host, port), timeout=2):
            return True
    except OSError:
        return False

def main():
    deadline = time.time() + TIMEOUT
    while time.time() < deadline:
        still_down = [name for name, h, p in DEPS if not reachable(h, p)]
        if not still_down:
            print("[wait] all dependencies reachable")
            break
        print(f"[wait] waiting on: {', '.join(still_down)}")
        time.sleep(INTERVAL)
    else:
        if still_down:
            print(f"[wait] TIMEOUT still down: {still_down}")
        sys.exit(1)

    os.execvp("python", ["python", "-u", "client.py"])

if __name__ == "__main__":
    main()
```

Img 5. Validación existencia del wait_for.py

5. Prueba de integración

Para comprobar el funcionamiento global del sistema y verificar cómo respondía nuestro código intermediario, realizamos una inyección de datos simulando un sensor externo. Durante este proceso, mantuvimos un monitoreo en tiempo real de los registros del cliente Python ejecutando el comando docker logs -f mqtt-pg-client.

Inicialmente, el registro confirma un arranque seguro con el mensaje [wait] all dependencies reachable, lo que demuestra que nuestro script auxiliar retuvo el inicio del cliente hasta que PostgreSQL y Mosquitto estuvieron totalmente listos para recibir conexiones. Justo después, observamos el éxito principal de la prueba: el cliente intercepta la lectura simulada y confirma tanto la creación dinámica de la tabla como la persistencia del dato con la línea [db] stored in table 'sensors_temp'. Finalmente, la última sección del log revela un comportamiento muy valioso para aplicaciones reales, ya que el cliente entra en estado de espera ([wait] waiting on: postgres) y luego se reconecta automáticamente.

```
saraesc@saraserver:~$ docker logs -f mqtt-pg-client
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[mqtt] sensors/temp -> {'temp': 25.5}
[db] stored in table 'sensors_temp'
[wait] waiting on: postgres
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[wait] waiting on: postgres
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
```

Img 6.Prueba de integración

RESULTADOS

Para comprobar el funcionamiento global del sistema y verificar cómo respondía nuestro código intermediario, realizamos una inyección de datos simulando un sensor externo. Durante este proceso, revisamos los registros más recientes del cliente Python ejecutando el comando docker logs mqtt-pg-client --tail 10.

Al analizar esta salida, se puede evidenciar de forma muy clara el ciclo de la arquitectura. En la primera parte del registro,el cliente entró en estado de espera ([wait] waiting on: postgres) y luego retomó su ejecución al confirmar que los servicios estaban listos[wait] all dependencies reachable). Finalmente, en las últimas dos líneas observamos el éxito definitivo de la prueba: el cliente intercepta la lectura simulada ({'temp': 25.5}) y confirma tanto la creación dinámica de la tabla como la persistencia del dato en la base de datos PostgreSQL con la línea [db] stored in table 'sensors_temp'.

```
saraesc@saraserver:~$ docker logs mqtt-pg-client --tail 10
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[wait] waiting on: postgres
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[mqtt] sensors/temp -> {'temp': 25.5}
[db] stored in table 'sensors_temp'
saraesc@saraserver:~$
```

Img 7. Funcionamiento del sistema completo.

Para confirmar físicamente que la persistencia de los datos fue exitosa, ingresamos al contenedor de PostgreSQL y realizamos una consulta directa a la tabla que el cliente Python generó automáticamente mediante el siguiente comando, docker exec -it postgres psql -U app_user -d iot_db -c "SELECT * FROM sensors_temp;". A continuación se muestra la salida obtenida en la terminal de la base de datos.

```
 id | temp |          received_at
----+------+-------------------------------
  1 | 25.5 | 2026-10-14 21:50:47.203227+00
  2 | 25.5 | 2026-10-21 21:44:30.754463+00
(2 rows)

saraesc@saraserver:~$
```

Img 8. Base de datos.

Como se puede observar en la salida, la base de datos estructuró correctamente la información en las columnas diseñadas (id, temp y received_at). Se puede evidenciar que la consulta nos arrojó dos registros distintos, evidenciando cómo el sistema asigna automáticamente un identificador incremental (ID 1 y 2) para cada evento. Además, podemos ver que el sistema almacenó una lectura antigua del 14 de octubre junto con la nueva inyección del 21 de octubre, asignándoles a ambas estampas de tiempo extremadamente precisas.

PREGUNTAS

T1. ¿Por qué montamos la contraseña (passwd) como un archivo en lugar de hornearla (bake) dentro de la imagen Docker?

Si se integra psswd directamente en la imagen de Docker, cualquier persona que inspeccione las capas de la imagen o le haga un pull podrá extraer las contraseñas. Al montarla en tiempo de ejecución (bind-mount), el secreto se mantiene en tu sistema anfitrión (host), fuera del historial de Docker, permitiendo además cambiar la contraseña en el futuro sin tener que recompilar todo el contenedor.

T2. ¿Por qué la imagen de PostgreSQL ignora la variable POSTGRES_PASSWORD después de la primera vez que se ejecuta?

El script de inicio (entrypoint) de PostgreSQL solo utiliza esas variables para crear un clúster vacío la primera vez. Una vez que detecta que ya existen datos en /var/lib/postgresql, salta la fase de inicialización y arranca el servidor directamente, ignorando las variables de entorno. Si necesitas cambiar la clave después, debes entrar con psql y usar el comando SQL ALTER USER.

T3. ¿Por qué el cliente usa un script wait_for.py en lugar de las opciones nativas --link o depends_on de Docker?

La opción --link está obsoleta y solo ordena el encendido, no garantiza que el servicio esté listo para recibir datos. depends_on (con comprobaciones de salud) es la herramienta correcta, pero solo existe para docker-compose, no para comandos de docker run. El script interno evalúa la disponibilidad real del puerto TCP para evitar errores de conexión prematura.

T4. ¿Qué impacto en rendimiento tiene ejecutar CREATE TABLE IF NOT EXISTS con cada mensaje que llega?

Para PostgreSQL, esto implica analizar la sentencia y hacer una búsqueda rápida en su catálogo, lo cual es muy barato computacionalmente. Es imperceptible con pocos mensajes por segundo. Sin embargo, para un sistema de alto tráfico, lo ideal sería guardar el estado en la memoria caché del cliente de Python para crear la tabla solo la primera vez que se detecta un tópico nuevo.

T5. ¿Por qué configuramos protocol=mqtt.MQTTv5 en el cliente Python?

La versión 5 de MQTT incluye mejoras modernas como códigos de razón (reason codes) para errores, mejor manejo de reportes y evita recibir mensajes en el momento exacto de suscribirse. Como Mosquitto 2.x lo soporta por defecto, aprovechamos estas ventajas.

T6. ¿Por qué el cliente abre y cierra una conexión a la base de datos por cada mensaje en lugar de usar una sola conexión abierta todo el tiempo?

Por simplicidad, si la base de datos se reiniciara y tuvieras una única conexión abierta, esta quedaría "muerta" y el código fallaría en silencio. Para entornos de producción profesionales, la solución correcta es implementar un "Connection Pool" (psycopg2.pool.SimpleConnectionPool), que recicla conexiones y verifica si están activas antes de usarlas.

C1. El puerto 1883 del bróker está expuesto hacia afuera. ¿Cuál es el riesgo y cómo lo solucionarías en un entorno de producción real?
El riesgo es que cualquier máquina en la red local o internet intente conectarse al bróker para adivinar las credenciales (ataque de fuerza bruta). Las mitigaciones profesionales incluyen: no publicar el puerto y requerir VPN, habilitar TLS (certificados de seguridad en el puerto 8883) y bloquear temporalmente las IPs que fallen repetidamente (fail2ban).

C2. El cliente deriva el nombre de la tabla directamente del tópico MQTT. ¿Qué riesgo de inyección existe?
Un atacante podría enviar un tópico malicioso como foo"; DROP TABLE sensors_temp; para intentar borrar la base de datos. Para blindar el sistema, se debe validar el nombre de la tabla usando expresiones regulares (Regex) para aceptar solo letras y números, y usar utilidades de escape seguras como psycopg2.sql.Identifier en lugar de inyectar el texto crudo (f-strings) en la consulta SQL.

C3. ¿Qué pasa con los mensajes que están en camino si el script de Python se bloquea justo antes de insertarlos en la base de datos?
Por defecto, usamos Calidad de Servicio 0 (QoS 0) en MQTT, lo que significa que el mensaje se perderá para siempre. Si se sube a QoS 1, Mosquitto lo reintentará, pero Python podría terminar guardándolo dos veces (duplicados). La solución más sólida es usar QoS 1 sumado a una restricción SQL (Unique ID o ON CONFLICT DO NOTHING) para garantizar que el dato se guarde una única vez.

C4. Como la base de datos no expone puertos externos, ¿cómo puedes ejecutar una consulta SQL desde tu computadora sin abrir el puerto permanentemente?
Ejecutando el comando docker exec -it postgres psql -U youruser -d yourdb -c '...' directamente sobre el contenedor en ejecución. También puedes levantar un contenedor temporal efímero en la misma red appnet solo para hacer la consulta y que se destruya al terminar.

C5. Si el sistema escala a 1000 sensores enviando datos cada segundo, ¿qué colapsará primero?
El puente de Python colapsará. PostgreSQL y Mosquitto pueden manejar 1000 msgs/s sin problemas, pero el proceso de abrir y cerrar la conexión a la base de datos (handshake) por cada lectura individual tomará demasiado tiempo. Se soluciona procesando los insertos en lotes (batch inserts) y usando un Connection Pool o librerías asíncronas (aiomqtt + asyncpg).

C6. ¿Por qué la política --restart always es peligrosa durante un despliegue con código defectuoso?
Esta política obliga a Docker a reiniciar el contenedor sin importar la razón de la caída, creando un bucle de reinicios infinito que llena el sistema de logs basura. Es mucho más seguro utilizar unless-stopped o on-failure:5, la cual desiste de reiniciar después de 5 intentos fallidos consecutivos para que el administrador note el error.

III. TLS-Compose-Docker
METODOLOGÍA

Para dar inicio al laboratorio número 4, es importante tener claros los conceptos adicionales que este maneja. A diferencia del laboratorio 3, que se compone principalmente del broker MQTT, el cliente Python y la base de datos PostgreSQL, el laboratorio 4 adiciona una capa de seguridad muy importante en este tipo de transmisiones: el protocolo TLS.

Transport Layer Security, también conocido por sus siglas TLS, es un protocolo de seguridad ampliamente utilizado, diseñado para proporcionar privacidad durante las comunicaciones realizadas a través de una red. Este protocolo se basa principalmente en tres componentes: la encriptación, la autenticación y la integridad. La encriptación permite proteger la información transmitida, la autenticación verifica la identidad de las partes involucradas y la integridad garantiza que los datos no hayan sido modificados durante la transmisión. En términos generales, TLS inicia mediante una secuencia conocida como protocolo de enlace, durante este proceso se establece la versión de TLS que se utilizará, los algoritmos de cifrado compatibles, se verifica la identidad del servidor mediante certificados digitales y se generan las claves que posteriormente serán utilizadas para proteger la comunicación.

Partiendo de este concepto es que se realizan las variaciones importantes de este laboratorio, la primera de estas diferencias importantes corresponde a la creación y utilización de certificados digitales, mientras que en el laboratorio anterior el cliente podía conectarse directamente al broker MQTT, en este caso es necesario generar previamente una autoridad certificadora, una clave privada para el servidor y un certificado para Mosquitto.

```
salome@salomeserver:~/netproject$ cd broker
salome@salomeserver:~/netproject/broker$ ls -al
total 28
drwxrwxr-x 7 salome salome 4096 Oct 21 01:51 .
drwxrwxr-x 6 salome salome 4096 Oct 21 02:57 ..
drwxrwxr-x 2 salome salome 4096 Oct  1 01:41 ca_certs
drwxr-xr-x 2 root   root   4096 Oct 21 02:11 certs
drwxrwxr-x 2 salome salome 4096 Oct 21 02:08 config
drwxrwxr-x 2 1883   1883   4096 Oct 21 01:51 data
drwxrwxr-x 2 salome salome 4096 Oct  1 02:11 log
salome@salomeserver:~/netproject/broker$
```

Img 9. Contenido de la carpeta Broker, en la cual se puede visualizar el almacenamiento de los certificados TLS

Para realizar este proceso se utilizó OpenSSL, una herramienta ampliamente empleada para trabajar con criptografía, claves y certificados digitales. En primer lugar se creó una autoridad certificadora local o CA. Esta autoridad funciona como una entidad de confianza dentro del laboratorio y es la encargada de firmar el certificado que posteriormente utilizará el broker MQTT, después de esto se generaron la clave privada y el certificado correspondientes al servidor Mosquitto, de esta manera, cuando un cliente intenta establecer una conexión segura, Mosquitto presenta su certificado y el cliente puede verificarlo utilizando el certificado de la autoridad certificadora.

Uno de los aspectos importantes durante esta etapa fue la correcta identificación del servidor dentro del certificado. Debido a que los servicios funcionan dentro de Docker, el cliente Python se conecta al broker utilizando el nombre mosquitto, mientras que desde la máquina virtual también pueden realizarse conexiones utilizando localhost. Por esta razón, fue necesario considerar estos nombres al generar el certificado para evitar problemas de validación TLS.

```
X509v3 Subject Alternative Name:
    DNS:mosquitto, DNS:localhost, IP Address:127.0.0.1
```

Img 10. Aplicación de SAN para reconocer al broker mosquitto con diferentes nombres

Después de crear los certificados fue necesario modificar la configuración de Mosquitto, a diferencia de lo usado en el laboratorio 3 el broker funcionaba principalmente con MQTT convencional, mientras que en este laboratorio se agregó un listener seguro mediante el puerto 8883.

```
# === Plain listener ===
listener 1883 0.0.0.0
allow_anonymous false
password_file /mosquitto/config/passwd

# === TLS listener on 8883 ===
listener 8883 0.0.0.0
certfile /mosquitto/certs/server.crt
keyfile /mosquitto/certs/server.key
cafile /mosquitto/certs/ca.crt

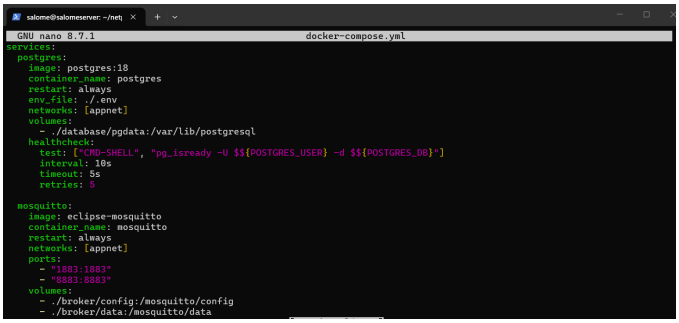

persistence_location /mosquitto/data/
log_dest stdout
```

Img 11. Ajuste de Mosquitto Config - Adición del puerto TLS

Sin embargo, cabe aclarar que el puerto 1883 continúa siendo utilizado para conexiones MQTT convencionales, mientras que el puerto 8883 se configuró para aceptar conexiones MQTT protegidas mediante TLS, para habilitar esta comunicación, Mosquitto debe tener acceso al certificado de la autoridad certificadora, al certificado del servidor y a su clave privada, adicionalmente se mantuvo la autenticación mediante usuario y contraseña, por lo que la seguridad del sistema abarca tanto los certificados que solicita TLS como la autenticación original que se realizaba

Con la finalidad de evitar que las personas que quieran replicar este laboratorio cometan los errores que encontramos sobre la marcha, se expondran durante el informe los errores y las soluciones encontradas para los mismos, durante la configuración del mosquitto se presentaron varios errores, en primera instancia se encontró una definición duplicada de persistence_location,se hicieron las eliminaciones correspondientes pero al actualizar el flujo simplemente indicaba que la repeticion se encontraba en una linea de codigo que no estaba siendo utilizada, por ende, para evitar estos conflictos, se elimino el archivo y se creo desde 0, permitiendo asi continuar con la configuración.

Otra diferencia importante respecto al laboratorio 3 corresponde a la utilización de Docker Compose, en el laboratorio anterior, los diferentes contenedores podían administrarse mediante comandos independientes, es decir, cada uno tenia su propia inicialización con docker run, sin embargo, en el laboratorio 4 se realizó la implementación de un docker compose, que los inicializa todos dentro de un mismo documento, asignandoles su imagen, nombre, numero de puerto, etc.



Img 12. Configuración de los diferentes servicios en el Docker Compose.

Dentro de este archivo se definen los tres servicios principales del sistema, postgreSQL, Mosquitto y el cliente Python encargado de recibir los mensajes MQTT y almacenarlos en la base de datos.

Con esto ya tenemos los componentes necesarios, solo falta unirlos y hacer que funcionen entre si, para ello se hace uso de una red interna denominada appnet , la cual permite que todos los servicios se encuentren entre ellos usando sus nombres, sin necesidad de conocer sus IPs internas.

Otro concepto nuevo incluido en el laboratorio 4 corresponde a los healthchecks, estos comprobadores permiten determinar si el servicio se encuentra tanto encendido como listo para trabajar, por ejemplo, revisa si la base de datos esta lista para recibir información o si por ejemplo mosquitto ya puede recibir conexiones

Un cambio respecto al laboratorio 3, y que resulta fundamental para el funcionamiento de las nuevas medidas de seguridad implementadas, corresponde a la configuración del cliente Python. Anteriormente, el cliente establecía una conexión MQTT convencional con el broker. En este laboratorio fue necesario modificar dicha conexión para utilizar el puerto seguro 8883 y el protocolo TLS. Para establecer esta comunicación, el cliente carga el certificado de la autoridad certificadora y lo utiliza como referencia para verificar que el certificado presentado por Mosquitto sea válido y haya sido firmado por una autoridad de confianza dentro del entorno del laboratorio. De esta manera, antes de iniciar el intercambio de información, el cliente puede comprobar la identidad del broker al que se está conectando.

```
# TLS configuration
client.tls_set(
    ca_certs=MQTT_CA_CERT,
    cert_reqs=ssl.CERT_REQUIRED,
    tls_version=ssl.PROTOCOL_TLS_CLIENT,
)

client.on_connect = on_connect
client.on_message = on_message

while True:
```

Img 13. Reescritura del client.py con la finalidad de añadir la comprobación del broker con los certificados.

Una vez establecida correctamente la conexión TLS, el cliente continúa realizando la autenticación mediante usuario y contraseña, al igual que en el laboratorio anterior, y posteriormente se suscribe al tópico para recibir los valores publicados.

Una vez aplicados todos los pasos anteriores, se realizaron pruebas completas del sistema mediante la publicación de valores de temperatura en el tópico sensors_temp.

RESULTADOS

Para confirmar el funcionamiento del laboratorio se realzaron pruebas de publicación y suscripción con el broker mosquitto, además de revisar si en la base de datos de postgresSQL se estaba almacenando la información sin inconvenientes

Los mensajes fueron enviados al broker Mosquitto utilizando el puerto seguro 8883 y el certificado de la autoridad certificadora, posteriormente se revisaron los registros del cliente Python, donde se pudo confirmar primero la conexión exitosa mediante TLS, después la suscripción al tópico y finalmente la recepción de los valores publicados.

```
salome@salomeserver:~$ mosquitto_pub -h localhost -p 8883 \--cafile broker/certs/ca.crt \-u salome -P salome \-t sensors
/temp -m '{"temp": 21.6}'
salome@salomeserver:~$ mosquitto_pub -h localhost -p 8883 \--cafile broker/certs/ca.crt \-u salome -P salome \-t sensors
/temp -m '{"temp": 21.6}'
salome@salomeserver:~$
```

Img 14. Publicación MQTT.

```
salome@salomeserver:~$ mosquitto_sub -h localhost -p 8883 --cafile broker/certs/ca.crt -u salome -P salome -t sensors/te
mp -v
sensors/temp {"temp": 21.6}
```

Img 15. Suscripción MQTT

Los datos recibidos fueron procesados por el cliente y almacenados en la tabla sensors_temp de PostgreSQL.

```
BD1=# SELECT * FROM sensors_temp;
 id | temp |          received_at
----+------+-------------------------------
  1 | 26.3 | 2026-10-14 19:39:26.3004+00
  2 | 22.1 | 2026-10-21 02:17:11.845874+00
  3 | 22.1 | 2026-10-21 02:20:48.308896+00
  4 | 22.1 | 2026-10-21 02:21:10.870475+00
  5 | 22.1 | 2026-10-21 03:00:58.717753+00
  6 | 23.4 | 2026-10-21 23:43:49.202235+00
  7 | 23.4 | 2026-10-21 23:45:58.978248+00
  8 | 23.4 | 2026-10-21 23:46:12.69622+00
  9 | 25.4 | 2026-10-21 23:46:22.782413+00
 10 | 25.4 | 2026-10-21 23:53:17.090795+00
 11 | 25.4 | 2026-10-21 23:53:49.389671+00
 12 |   27 | 2026-10-21 23:54:16.81074+00
 13 |   27 | 2026-10-22 00:07:18.840257+00
 14 |   28 | 2026-10-22 00:07:26.974618+00
 15 | 22.5 | 2026-10-22 00:12:13.92184+00
 16 | 21.6 | 2026-10-22 00:26:42.381756+00
 17 | 21.6 | 2026-10-22 02:47:44.455169+00
 18 | 21.6 | 2026-10-22 02:48:01.446685+00
(18 rows)

BD1=#
```

Img 16. Almacenamiento en la base de datos de las publicaciones MQTT

De esta manera se verificó que el flujo estuviera funcionando correctamente

En comparación con el laboratorio 3, la función principal del sistema se mantiene, ya que permite recibir información mediante MQTT y almacenarla posteriormente en PostgreSQL. Sin embargo, el laboratorio 4 mejora considerablemente la seguridad del flujo de comunicación. La incorporación de TLS permite proteger la información transmitida y verificar la identidad del broker mediante certificados digitales, mientras que Docker Compose facilita la administración de todos los servicios desde una única configuración.

PREGUNTAS

T1. ¿Por qué el campo Common Name (CN) ya no se utiliza para la verificación del nombre del host?
El campo Common Name dejó de ser la referencia principal para verificar nombres de host porque fue pensado para identificar un solo nombre de referencia , mientras que en algunos casos un mismo certificado puede necesitar ser válido para varios nombres DNS o direcciones IP, en el laboratorio se puede observar esto con mosquitto. Por esta razón se utiliza la extensión Subject Alternative Name, o SAN, que permite incluir varias identidades, en el ejemplo con mosquitto, este puede identificarse como mosquitto, localhost o 127.0.0.1.

T2. ¿Por qué el certificado del servidor necesita CA:FALSE en basicConstraints?
El certificado del servidor debe tener CA:FALSE porque su función es únicamente identificar al servidor Mosquitto y no actuar como autoridad certificadora. Si tuviera permisos de CA, podría utilizarse para firmar otros certificados, funcionalidad que no es necesaria en esta aplicación, por ende el certificado queda limitado a su propósito de autenticación del servidor y reducir los riesgos de seguridad

T3. ¿Por qué se utiliza una clave de sesión simétrica después del handshake asimétrico?
TLS utiliza criptografía asimétrica principalmente durante el establecimiento inicial de la conexión, ya que permite autenticar al servidor y establecer de forma segura los elementos necesarios para generar las claves de sesión. Sin embargo, la criptografía asimétrica es mucho más costosa computacionalmente. Por esta razón, una vez terminado el enlace, la comunicación utiliza cifrado simétrico, que es mucho más rápido y eficiente para proteger el tráfico de datos.

T4. ¿Por qué el control de salud de Docker Compose funciona, pero docker run no ofrece el mismo comportamiento?
En el caso de Docker Compose, este conoce la relación entre todos los servicios definidos en el archivo de configuración y puede establecer como manejan las dependencias entre ellos, lo que permite esperar a que un servicio sea reportado como saludable antes de iniciar otro que depende de él. En cambio, docker run ejecuta cada contenedor de manera individual y no administra de forma automática ese tipo de relaciones entre varios servicios..

T5. ¿Por qué se debe escribir $$ en lugar de $ en los healthchecks de Docker Compose?
Docker Compose interpreta las variables antes de que el comando llegue al contenedor. Si se utiliza un solo signo $, Compose puede intentar sustituir la variable utilizando el entorno de la máquina anfitriona. Al escribir $$, se indica que debe conservarse un signo $ para que la variable sea interpretada posteriormente dentro del contenedor. De esta manera, el healthcheck utiliza correctamente las variables de entorno propias del servicio.

T6. ¿Por qué nc -z es suficiente para el healthcheck básico del broker?
Netcat permite verificar si Mosquitto está escuchando en un puerto TCP. Si el puerto está abierto, significa que el proceso del broker se encuentra activo y pudo crear correctamente su listener. Esta comprobación no permite hacer validaciones de otros compuestos como por ejemplo TLS, usuarios, contraseñas ni permisos MQTT, pero sí permite determinar si el servicio está disponible a nivel de red.

C1. ¿Por qué una autoridad certificadora propia es aceptable en este laboratorio pero no siempre en producción?
En este laboratorio se tiene control tanto sobre el broker como sobre los clientes, por lo que es posible distribuir manualmente el certificado de la autoridad certificadora y configurarlo como confiable. En un sistema público o con muchos dispositivos, este procedimiento sería más difícil de administrar. Por esa razón, los servicios públicos suelen utilizar autoridades certificadoras reconocidas. Sin embargo, una CA privada también puede utilizarse en producción en redes empresariales, IoT o sistemas industriales si existe un mecanismo seguro de distribución y administración de certificados.

C2. Se mantienen los puertos 1883 y 8883. ¿Cuál es el riesgo y cuándo debería eliminarse el 1883?
El puerto 1883 permite MQTT sin cifrado, por lo que un cliente externo que cuente con la información necesaria podría transmitir credenciales o datos sensibles sin protección, manteniendo asi una superficie de ataque. Una vez que todos los clientes hayan sido migrados a TLS, se debe deshabilitar el puerto 1883 o restringirlo

C3. ¿Qué ocurre si un atacante obtiene la clave privada de la autoridad certificadora ca.key?
Si un atacante obtiene la clave privada de la CA, podría generar certificados falsos que serían considerados válidos por los clientes, esto conlleva a una suplantación de la identidad del broker y permitirá los ataques de intermediarios. Por esta razón, ca.key con permisos estrictos y fuera de los servidores operativos

C4. ¿Qué fallos no detecta un healthcheck basado únicamente en nc -z?
Una comprobación TCP solo confirma que el puerto está abierto y preparado para la escucha, no cumple con detectar certificados vencidos, errores en las claves, problemas con los SAN, credenciales incorrectas, permisos MQTT, errores de autorización ni fallos en la persistencia. Por eso, aunque el servicio aparezca como saludable, aún podría existir un problema en capas superiores

C5. depends_on con service_healthy controla el arranque, pero ¿qué pasa si el broker falla durante la ejecución?

Las funciones explicitas de las condiciones de salud actuan principalmente durante el proceso de inicio de los contenedores, su trabajo es evitar que un servicio dependiente de otros, como el cliente Python, comience a ejecutarse antes de que Mosquitto haya sido reportado como saludable. Sin embargo, una vez que todos los servicios ya están en funcionamiento, esta condición no continúa supervisando permanentemente al broker ni reinicia automáticamente al cliente si Mosquitto falla durante la ejecución.

C6. ¿Por qué montar los certificados como solo lectura no protege completamente server.key?
La opción de solo lectura evita que el contenedor modifique o elimine el archivo, pero no impide que lo lea. Mosquitto necesita leer server.key para establecer conexiones TLS. Por esta razón, si un atacante compromete el contenedor y obtiene permisos suficientes, podría copiar la clave aunque el volumen esté configurado como solo lectura.

IV. CONCLUSIONES

- La implementación de Docker permitió comprender que una arquitectura IoT segura no depende únicamente de que sus componentes funcionen, sino de cómo se aíslan, se comunican y controlan el acceso entre sí. La red interna, la restricción de conexiones anónimas en Mosquitto y la no exposición directa de PostgreSQL ayudaron a reducir la superficie de ataque, mientras que el laboratorio 4 reforzó esta seguridad mediante TLS, mostrando que proteger la comunicación también implica cifrar los datos, autenticar al servidor y verificar su identidad mediante certificados digitales.
- El desarrollo del cliente Python fue uno de los aprendizajes más relevantes del laboratorio, ya que permitió integrar MQTT con PostgreSQL aun cuando ambos servicios se encuentran ejecutándose en contenedores diferentes. La implementación de este flujo de recepción, procesamiento y almacenamiento de datos permitió comprender de manera más concreta cómo una arquitectura de este tipo puede aplicarse en entornos ajenos a la academia, donde la información proveniente de sensores debe ser transmitida, procesada y almacenada de forma continua y organizada.
- La incorporación de Docker Compose, los healthchecks y las dependencias permitió entender que un servicio no necesariamente está listo para operar únicamente porque su contenedor se haya iniciado. La correcta coordinación entre los componentes es fundamental para garantizar que el sistema funcione de forma estable, reduzca fallos durante el arranque y pueda mantener una comunicación confiable entre sus diferentes servicios.
- Los errores presentados durante los laboratorios fueron parte importante del aprendizaje, ya que permitieron comprender mejor el funcionamiento real de la arquitectura. Los problemas con certificados, bind mounts, healthchecks y permisos de PostgreSQL demostraron que muchas fallas suelen deberse al poco conocimiento y manejo de los comandos encargados de la interacción entre diferentes capas del sistema.

V. REFERENCIAS

[1] https://www.redhat.com/es/topics/containers/what-is-docker

[2] Repositorio de GITHUB

https://github.com/Salome05/netproject
https://github.com/saraescobarsg/iot-lab

[3] Eclipse Foundation, “Eclipse Paho MQTT Python Client,” *Eclipse Paho Documentation*. [Online]. Available: https://eclipse.dev/paho/clients/python/docs/. [Accessed: Oct. 21, 2026].

[4] Docker, Inc., “Docker Compose Quickstart,” *Docker Documentation*. [Online]. Available: https://docs.docker.com/compose/gettingstarted/. [Accessed: Oct. 21, 2026].

[5] Docker, Inc., “Bind mounts,” *Docker Documentation*. [Online]. Available: https://docs.docker.com/engine/storage/bind-mounts/. [Accessed: Oct. 21, 2026].

[6] GitHub, Inc., “Removing sensitive data from a repository,” *GitHub Docs*. [Online]. Available: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository. [Accessed: Oct. 21, 2026].

LINK AL REPOSITORIO DE GITHUB

Salome05/netproject: Repository to collaborate on networks project